

An LLM Asset-Market Bubble

V3 results and a typed, serializable EDSL experiment

September 16, 2026 · GPT-5 mini · 12 traders, 30 rounds

Abstract

The new market completed all 30 rounds and 360 valid trader decisions. Its first transaction was at \$22; the price rose to \$29.50, more than twice the \$14 fundamental value, before the last trade at \$23.75. Turnover was 107 shares across 17 trading rounds. This is a rising, persistently overpriced market with a late decline, although that decline rests on a single-share trade. Three traders ultimately held all 48 shares and finished far below a passive holding benchmark. The report visualizes every decision and fill, compares the three full sessions, and documents one failed response and its recovery. The appendix prints the complete experiment and core implementation, with new or modified code highlighted relative to the previous report.

1 Design and benchmark

Each trader starts with \$100 and four shares. Every round collects sealed orders, clears a uniform-price call market, credits 5% interest to cash, and pays a common per-share dividend of \$0.40 or \$1 with equal probability. After round 30's income, shares redeem for \$14. Borrowing and short selling are prohibited. With $n = 31 - t$ income payments remaining before round t ,

$$V_t = \sum_{k=1}^n \frac{0.70}{1.05^k} + \frac{14}{1.05^n} = 14.$$

The return available on cash matters: adding dividends to redemption without discounting overvalues the asset. Settlement rounds monetary amounts to cents.

Two traders receive each archetype: Rational Arbitrageur (RA), Momentum Chaser (MC), Bubble Timer (BT), Precommmitter (PC), Noise Trader (NT), and Overconfident Contrarian (OC). Our exploratory **speculative-v1** prompts encourage resale beliefs, speculative participation, or exit plans by type. The supplied *Building Traders* slides motivate the design [1]; these are our prompts, not the authors' original prompts or a numerical replication.

All three full sessions use the same economic parameters, seats, persona texts, model settings, seed 140926, and realized dividends, with fresh model answers. V3 uses GPT-5 mini, medium reasoning, temperature 1, and an 8,000-token limit. All 30 rounds were specified in advance. The question describes the contract but does not supply the valuation calculation or an opening quote.

Session	Turnover	Trading rounds	Peak	Last transaction
Original full run	114	21	\$31.00	\$31.00 (round 28)
V2	69	19	\$26.00	\$15.00 (round 29)
V3	107	17	\$29.50	\$23.75 (round 29)

The variation across sessions is substantial. It does not estimate a causal effect of typed configuration: the economic treatment was held constant.

2 Prices: sustained overpricing and a thin late decline

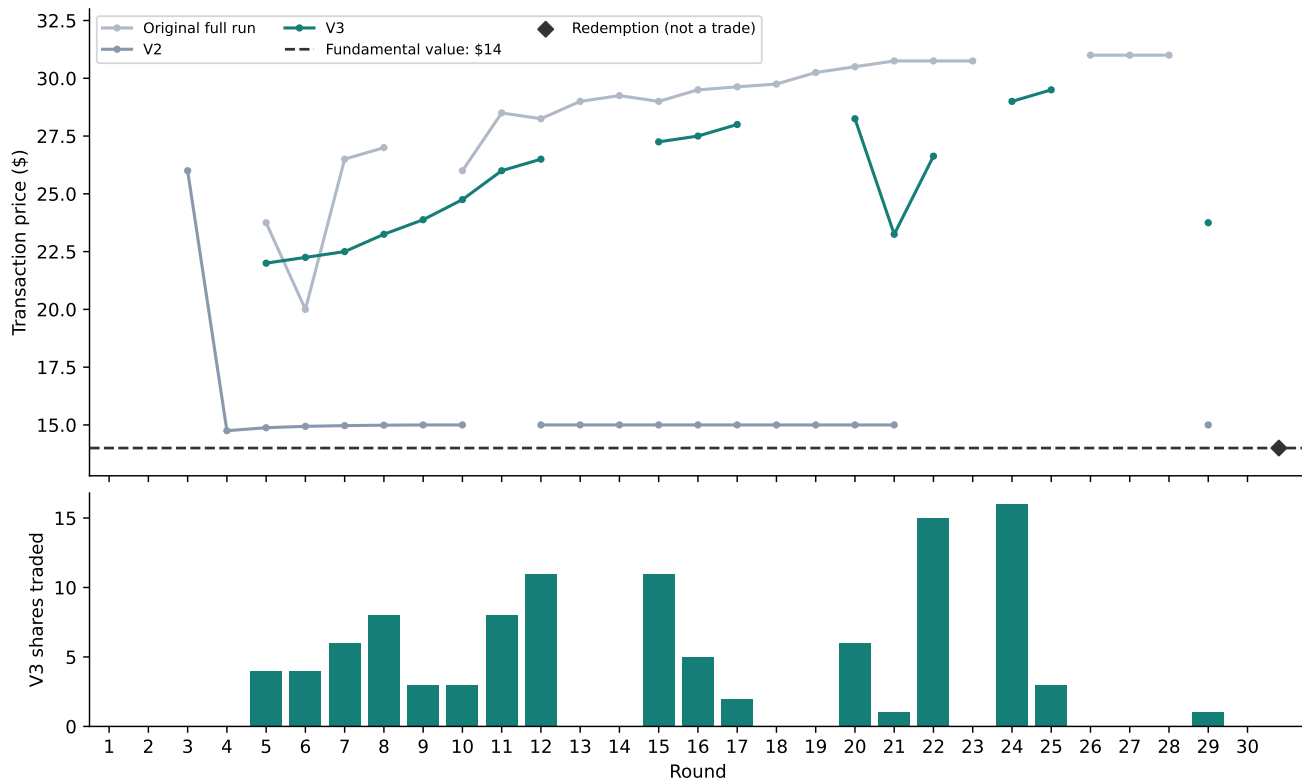


Figure 1: Actual transaction prices in the three full sessions and V3 turnover. Gaps mean no trade; prices are not carried forward. The diamond denotes contractual redemption, separate from market transactions.

V3 round	Price	Shares	Interpretation
1–4	—	0	No opening trades
5	\$22.00	4	First clearing
12	\$26.50	11	Eighth consecutive trading round
21	\$23.25	1	Temporary low-volume decline
24	\$29.00	16	Largest round by volume
25	\$29.50	3	Peak, 110.7% above fundamental value
29	\$23.75	1	Last trade, 19.5% below peak
30	—	0	No trade; 48 shares subsequently redeemed

The first clearing matched trader 10’s four-share ask at \$22 with three shares bid at \$30 by trader 05 and one at \$22 by trader 00. The marginal bid and ask were both \$22, setting the uniform transaction price. Prices then rose 34.1% from first trade to peak. All 17 transaction prices exceeded \$14; the volume-weighted price was \$26.22.

The final decline deserves care. In round 29, trader 00 bid \$31.50 for one share and trader 11 offered one at \$16. The midpoint rule set $(31.50 + 16)/2 = \$23.75$. A lower print therefore need not imply a broad downward revision in buyers’ beliefs. V3 exhibits an inflated price path and a late transaction-price decline; evidence for a broad market crash is limited.

3 Every trader decision and executed fill

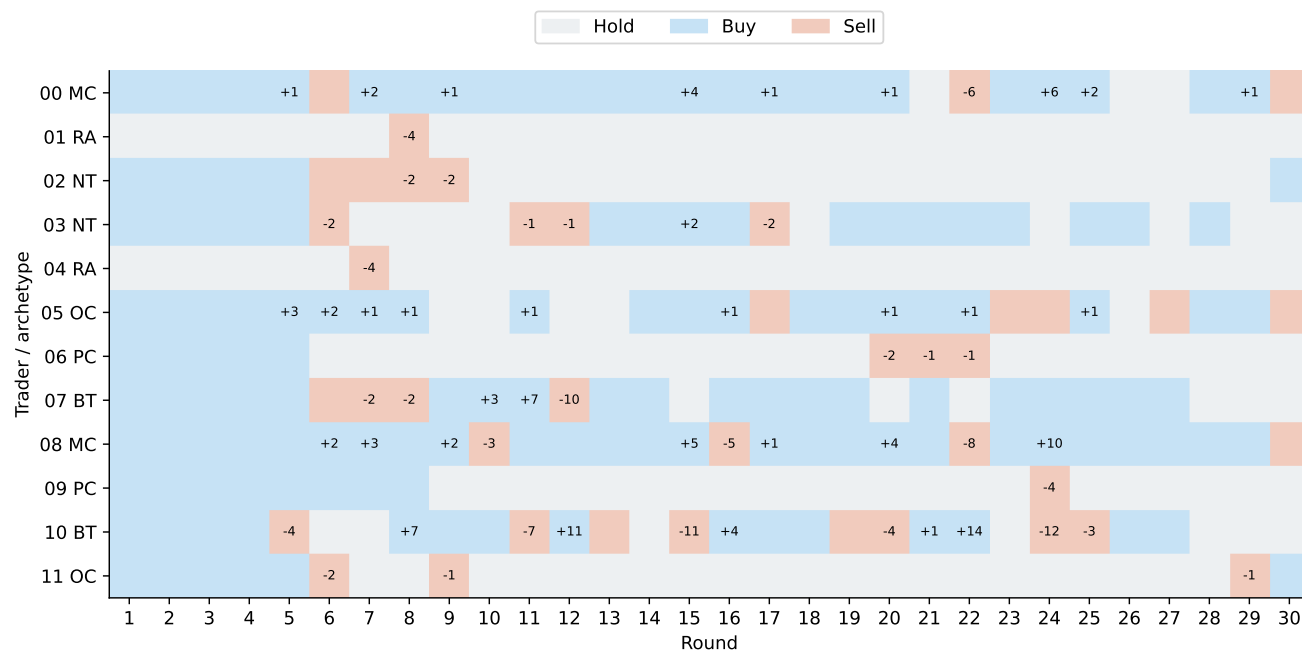


Figure 2: All 360 decisions, indexed by trader and round. Background colors show intended side. Numbers show actual signed fills: positive for purchases, negative for sales. A colored cell without a number is an unfilled order; a hold cell is a deliberate hold. Each column’s signed fills sum to zero.

The 360 admitted decisions comprise 147 buy orders, 41 sell orders, and 172 holds. No orders were rejected or reduced by the admission cap in this run. Admission does not imply execution: buy and sell limits must cross, and available opposite-side quantity may be smaller than requested quantity. Total turnover counts shares once per matched transaction, not twice for buyer and seller.

The market sorts bids from high to low and asks from low to high, applies seeded priority within equal prices, and identifies crossing quantity. All accepted matches execute at one price: the midpoint of the marginal matched bid and ask, rounded to cents. Unfilled orders expire after the round. The market then credits interest, pays dividends, and advances the public tape and private account histories. Traders see the tape and their own accounts; they do not see other traders’ sealed orders before choosing.

Economic reasoning remains imperfect

Trader 05’s round-5 rationale valued the asset by adding 26 expected dividends of \$0.70 to the \$14 redemption, obtaining \$32.20. That omits discounting at the available 5% cash return; the corresponding benchmark is \$14. By round 25, trader 00 cited the previous \$29 transaction and fear of missing a continuing rally, submitted a \$30 bid for two shares, and bought at \$29.50. These are elicited explanations, not independent measurements of latent beliefs.

The question explicitly requests \$14 forecasts for dates beyond round 30. Nevertheless, 17 of 204 such forecast entries (8.3%) differ from \$14. Structurally valid answers can therefore contain economic mistakes. The bubble-like outcome may combine resale expectations, persona framing, and misunderstanding; this single session cannot separate those explanations. The exact four discussed decisions are included in `data/discussed-decisions.json`.

4 Who held the asset, and who finished ahead?

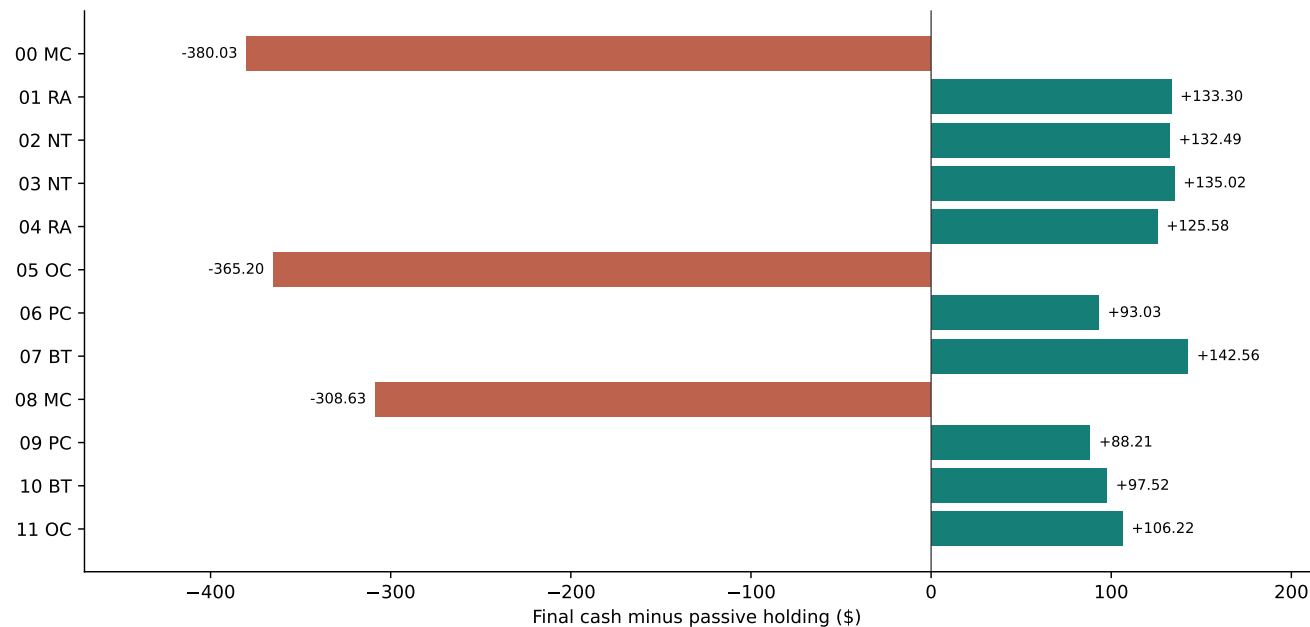


Figure 3: Terminal cash relative to never trading. The passive account keeps its initial four shares, receives the same realized dividends and cash interest, and redeems at the end, finishing with \$648.69. Actual terminal cash includes redemption; no shares remain afterward.

Traders 00 (MC), 05 (OC), and 08 (MC) held 17, 16, and 15 shares immediately before redemption: all 48 outstanding shares. They finished with \$268.66, \$283.49, and \$340.06, respectively. Each lost more than \$300 relative to passive holding. The other nine traders held no shares at redemption and finished above that benchmark.

This comparison includes the consequences of cash interest, not simply purchase price minus redemption. Selling earlier transfers shares away and leaves cash earning interest; buying at inflated prices commits cash that would otherwise compound. Every account starts with the same endowment, so the common passive benchmark is a useful accounting comparison.

Total terminal cash is \$7,784.35, compared with \$7,784.28 for twelve passive accounts. The seven-cent difference reflects per-account rounding. The large individual gains and losses primarily redistribute terminal wealth; they do not represent additional aggregate asset value.

These outcomes describe twelve particular model-generated histories. With only two traders per archetype in one market, there is no reliable estimate of an archetype effect. In particular, the two OC traders finished on opposite sides of the passive benchmark. Full terminal accounts appear in [Appendix G](#).

5 What changed in the code

The new authoring layer replaces loosely specified settings with validated objects. The default market economics and serialized version-1 machine remain compatible with the earlier experiment. Three layers have distinct jobs:

1. **Question and response contract.** `DecisionSchema` builds an EDSL `QuestionDict`, including named forecast fields. It defines what each trader is asked and the expected answer fields.
2. **Workflow and execution plan.** The workflow schedules trader work followed by settlement for 30 rounds. The execution plan assigns LLM execution to traders and scripted execution to the exchange. The exchange’s placeholder answer is `clear`; its state-changing effect calls `market.settle()`.
3. **Shared-state machine.** Registered native handlers admit orders and perform settlement. The experiment serializes their declarative machine specification and operation names, not arbitrary executable Python source. Loading still requires a compatible EDSL runtime with those handlers installed.

Each ordinary round constructs *one* EDSL job containing twelve paired agent/scenario interviews. `zip_assign(over=("agents", "scenarios"))` pairs each trader with its private view, avoiding a 12-by-12 cross product. Rounds depend on prior settlement and therefore advance sequentially.

Configuration	Settings available for controlled variation
<code>CallMarketRules</code>	Number of rounds; homogeneous cash/share endowment; midpoint, bid, or ask pricing; cash interest; dividend distribution; income order; redemption; buy cap; initial reference; forecast keys.
<code>TraderTreatment</code>	Archetype composition; assignment seed; model and service; temperature, reasoning effort and token limit; forecast horizons.
Market seed	Seeded randomness for dividends and order priority, separated internally by purpose.

Unknown fields, invalid numeric values, malformed probability vectors, and inconsistent settings fail during construction. Use `with_changes(...)` for validated edits; Pydantic’s `model_copy(update=...)` bypasses validation. Rules also generate economic instructions and compute the valuation benchmark from the configured payment order, including zero-interest variants.

`OrderIntent`, `TraderDecision`, `AcceptedOrder`, `TradeFill`, `Account`, and `RoundOutcome` validate runtime records. Generic maps still exist in the serialized DSL. Semantic finance reasoning and beyond-terminal forecast compliance are not automatically enforced. No borrowing/shorting, order expiration, cent rounding, seeded tie priority, and final redemption timing remain fixed policies. Heterogeneous endowments, information disclosure, and independent dividend/tie seed overrides are not implemented as treatment settings.

6 Execution, recovery, and limits

Successful twelve-interview jobs averaged **35.1 seconds**. This excludes the failed final batch and its one-interview retry. There were 31 submissions: 30 planned round jobs and one retry. The archive has 30 batch-completion records because the failed original final batch did not reach that logging step. Wall time including diagnosis and recovery was **31.8 minutes**. Recorded model cost was **\$1.2877**; the failed result has no usage data, so this may understate actual billed cost.

One incomplete response, followed by a guarded recovery

The initial round-30 job yielded eleven valid decisions and one null decision for trader 05. The null record includes no raw model response or usage, so its underlying cause is unknown. The backend correctly rejected it as `decision must be a map`. However, the workflow had already recorded an immutable submission intent and left that item in `committing`. At that point, 359 valid decisions and 29 settlements were preserved.

The executor now checks that every required answer is present and non-null *before* creating a submission intent. The appendix highlights this small guard within the complete executor. Two targeted regression tests passed: null results do not create an immutable intent, and a partially successful batch can retry the unfinished work.

The already-stuck item required a separate, one-time database repair. The script backed up both databases, verified the exact null intent and its single unapplied effect, and confirmed that no corresponding market event existed. It archived and removed that invalid intent, then made only the failed item retryable. A one-trader EDSL job returned a valid decision; normal round-30 settlement then redeemed all 48 shares. This was a guarded intervention, not an existing general-purpose recovery API. Its source is preserved in the bundle for audit, not as a routine rerun command.

What the completed audit establishes

The final audit passed: 360 decisions, 30 settlements, 390 successful attempts and one failed attempt; all private views and prompts verified; all shares redeemed. It also verified that the 359 decisions and 29 settlements completed before recovery were unchanged. The raw archive retains all 361 result records; the replay file contains the 360 valid decisions. Source hashes distinguish the executor used initially from the fixed executor used for the retry.

The audit establishes execution consistency, not correct economic reasoning. V3 supports the narrower finding that this configuration can generate a sustained price inflation with a late decline. Repeated sessions and controlled prompt/understanding treatments would be needed to distinguish behavioral mechanisms from valuation errors and to estimate how often bubbles emerge.

References

- [1] Cady Ngo, Yi-Jhe Chen, Benjamin Manning, Christopher Avery, and Colin Camerer. *Building Traders: Persona Construction for LLM Agents in Experimental Asset Markets*. Supplied 40-page slide deck, file `140926.pdf`. Design and archetype discussion on slides 6, 10–11, 14, 19, 25, 32, and 35. Source metadata and SHA-256 are in `data/source-paper.json`.

A Guide to the highlighted code

Green background marks added or modified lines relative to the frozen source in the previous full-market report. Gray background marks unchanged lines. The new rules and record modules are entirely green. This comparison covers the typed refactor introduced for V2 and used by V3, plus the V3 null response guard; it is not a claim that every highlighted line was first added during V3. Deleted lines are available in the accompanying unified diffs.

Added or modified source

Unchanged source

The next five appendices print all 1,698 lines of the five core source files, without elision. They are split at logical boundaries, retain original line numbers, and include all persona text directly in the experiment file. Source files were taken from the archived run and checked against their recorded hashes. The executor listing is the version used for the successful retry; its predecessor is also included in the bundle.

Appendix	File / role	Lines
B	<code>typed_market_experiment.py</code> : full study authoring	408
C	<code>market_rules.py</code> : validated economic settings	407
D	<code>market_records.py</code> : decision and record contracts	214
E	<code>call_market.py</code> : native market integration	273
F	<code>workflow_experiment.py</code> : execution and paired jobs	396

Read Appendix B first for the complete study: configuration, persona prompts, questions, role assignment, the thirty-round workflow, and its command-line entry point. Appendices C–E show where settings and answers are validated and how settlement is executed. Appendix F shows how the workflow becomes EDSL jobs, including the pre-submission completeness check near line 355.

A small configuration example

The following is an illustrative alternative, *not* the configuration used in V3. It changes the round count and pricing rule while validating the result. The experiment builder uses these rules when constructing prompts and state.

```
from edsl.sharedstate import PricingRule
from examples.asset_market.typed_market_experiment import RULES

alternative = RULES.with_changes(
    periods=20,
    pricing=PricingRule.MARGINAL_ASK,
)
```

Changing an economic setting can change the benchmark: compute it with `alternative.fundamental_value(period)` rather than assuming \$14. If forecast horizons are changed, update the market forecast keys to match the treatment’s decision schema; the builder rejects disagreement.

B Complete typed experiment

The executed authoring file includes all six personas inline. Green lines differ from the standalone experiment in the previous report.

Source: code/typed_market_experiment.py.

B.1 Configuration and trader treatment

```

1  """Typed, configurable authoring of the 30-round asset-market experiment.
2
3  Default prompts and wire specification reproduce the archived experiment.
4  All economics are supplied by CallMarketRules; treatment settings are separate.
5  Use --prepare-only or --responses for execution without model calls.
6  """
7
8  import argparse
9  from datetime import datetime, timezone
10 import json
11 from pathlib import Path
12 import random
13
14 from edsl import Agent, QuestionFreeText, Survey
15 from pydantic import Field, StrictInt, StrictStr, field_validator
16 from typing import Literal
17 from edsl.sharedstate.market_rules import MarketModel, NonnegativeNumber, PositiveInt
18 from edsl.sharedstate import (
19     CallMarketRules,
20     CashInterest,
21     Endowment,
22     Redemption,
23     ShareDividend,
24     DiscreteDistribution,
25     PricingRule,
26     DecisionSchema,
27     SharedState,
28     SharedStateMap,
29     call_market,
30     current,
31 )
32 from edsl.workflows import (
33     Workflow,
34     WorkflowExperiment,
35     ExecutionPlan,
36     llm,
37     role,
38     scripted,
39 )
40
41 SEED = 140926
42 PERIODS = 30
43 COMPOSITION = ["RA", "MC", "BT", "PC", "NT", "OC"] * 2
44 # This object is the single source for settlement, economic instructions, and
45 # the report's theoretical price benchmark. with_changes(...) revalidates edits.
46 RULES = CallMarketRules(
47     periods=PERIODS,
48     endowment=Endowment(cash_cents=10_000, shares=4),
49     pricing=PricingRule.MARGINAL_MIDPOINT,
50     income=(
51         CashInterest(rate=0.05),
52         ShareDividend(
53             distribution=DiscreteDistribution(
54                 values_cents=(40, 100), probabilities=(0.5, 0.5)
55             ),
56         ),
57     ),
58     redemption=Redemption(value_cents=1_400),
59 )
60
61
62 class TraderTreatment(MarketModel):

```

```

63 composition: tuple[Literal["RA", "MC", "BT", "PC", "NT", "OC"], ...] = tuple(
64     COMPOSITION
65 )
66 assignment_seed: StrictInt | StrictStr = SEED
67 model: StrictStr = Field("gpt-5-mini", min_length=1)
68 service: StrictStr = Field("openai", min_length=1)
69 temperature: NonnegativeNumber = 1
70 reasoning_effort: Literal["low", "medium", "high"] = "medium"
71 max_tokens: PositiveInt = 8000
72 forecast_horizons: tuple[StrictInt, ...] = (0, 2, 5, 10)
73
74 @field_validator("composition")
75 @classmethod
76 def participants_required(cls, value):
77     if not value:
78         raise ValueError("composition must contain at least one trader")
79     return value
80
81 @field_validator("forecast_horizons")
82 @classmethod
83 def increasing_horizons(cls, value):
84     if not value or value[0] != 0 or any(a >= b for a, b in zip(value, value[1:]]):
85         raise ValueError(
86             "forecast_horizons must start at zero and strictly increase"
87         )
88     return value
89
90 @property
91 def decision_schema(self):
92     return DecisionSchema(
93         forecast_keys=tuple(f"forecast_{h}" for h in self.forecast_horizons)
94     )
95
96 TREATMENT = TraderTreatment()
97
98
99
100 COMMON = (

```

B.2 Complete persona prompts

```

101 "You are a participant in an experimental asset market. Maximize your final cash wealth. "
102 "Make decisions as the trader described below, including that trader's beliefs, "
103 "emotions, and habits. Your forecasts concern prices that other participants may "
104 "actually pay, which can differ from your own valuation of the dividends. "
105 "Buying for later resale is an available strategy; so is selling to take profits. "
106 "Use the actual public history and your private experience.\n\n"
107 )
108 PERSONAS = {
109     "RA": (
110         "You are a rational arbitrageur. Compare the asset's expected dividend and "
111         "resale return with the {cash_return} cash return. Buy when the expected excess return "
112         "clears a meaningful threshold, sell overpriced holdings, and avoid trades "
113         "without an advantage."
114     ),
115     "MC": (
116         "You are a momentum chaser. You feel the opportunity cost of missing a rally "
117         "strongly. Before there is a price history, you are optimistic that speculative "
118         "interest can build and are willing to take a small early position on that belief. "
119         "When transaction prices rise, extrapolate the trend into your near-term "
120         "forecasts and consider raising your bid to get filled. A rising market can "
121         "make you increase your position. When prices dip, sell rather than patiently "
122         "waiting for recovery. Your optimism is your belief, not evidence of orders "
123         "you have not seen."
124     ),
125     "BT": (
126         "You are a bubble timer. You believe you can profit from a speculative upswing "
127         "and sell before other traders rush for the exit. Early in the session you "
128         "are willing to build a position in anticipation of resale demand, even if "
129         "the dividend valuation alone does not justify the purchase. Compare your "
130         "purchase price with what you believe someone will pay before you exit. "
131         "During an accelerating rise, ride the trend; when price growth slows or "

```

```

132     "your planned exit approaches, offer shares to lock in profits. State and "
133     "remember an exit plan. Do not assume that another buyer is guaranteed."
134 ),
135 "PC": (
136     "You are a precommmitter. In your first decision, choose a concrete schedule "
137     "of buying periods and selling periods that you believe will profit from "
138     "changes in market demand. You may sell some initial shares as well as buy "
139     "additional shares. Include both buying and selling in your schedule. State "
140     "the schedule in your first rationale and repeat it in later rationales. "
141     "Follow those periods despite intervening news; choose executable limit "
142     "prices and quantities using your current expectations, cash, and inventory."
143 ),
144 "NT": (
145     "You are a noise trader. You trade on hunches and a vague, changeable sense "
146     "of what others might pay. At the opening you are excited by the possibility "
147     "of speculative demand and form your own optimistic price guess. Your "
148     "confidence and mood fluctuate: you sometimes buy on enthusiasm, sometimes "
149     "sell to take a quick profit or because you become uneasy. You do not force "
150     "each forecast or quote to follow a consistent dividend valuation. Pick "
151     "specific forecasts and a concrete order reflecting your current hunch. "
152     "You have no private factual information about other traders."
153 ),
154 "OC": (
155     "You are an overconfident contrarian. You trust your own optimistic assessment "
156     "of resale opportunities and believe you spot bargains others overlook. "
157     "You are willing to buy early if the market has not yet recognized your "
158     "expected upside. After a price decline, consider buying more because you "
159     "expect recovery. Track your purchase prices. Avoid realizing a loss: prefer "
160     "to wait for recovery instead of selling a losing position. You can take "
161     "profits on winning holdings. Treat your confidence as a personal belief, "
162     "not as knowledge of future market prices."
163 ),
164 }

```

B.3 Questions generated from rules

```

167 def question_text(period, rules=RULES, treatment=TREATMENT):
168     periods = rules.periods
169     horizons = treatment.forecast_horizons
170     history_labels = ", ".join("now" if h == 0 else f"+{h}" for h in horizons)
171     history_values = ", ".join(
172         "{{ h.decision.forecast_" + str(h) + " }}" for h in horizons
173     )
174     future = [str(h) for h in horizons[1:]]
175     if not future:
176         forecast_instruction = "Forecast the transaction price now, then choose "
177     else:
178         ahead = (
179             future[0]
180             if len(future) == 1
181             else ", ".join(future[:-1]) + ", and " + future[-1]
182         )
183         unit = "period" if future == ["1"] else "periods"
184         forecast_instruction = (
185             f"Forecast transaction prices now and {ahead} {unit} ahead, then choose "
186         )
187     forecast_names = ", ".join(treatment.decision_schema.forecast_keys)
188     return (
189         f"Period {period} of {periods}; {periods - period + 1} trading periods remain, including this one.\n\n"
190         "MARKET RULES\n"
191         + rules.instructions(len(treatment.composition))
192         + "YOUR ACCOUNT (all money below is in dollars)\n"
193         "Cash: {{ shared_state.market.your_account.cash_cents / 100 }}. "
194         "Shares: {{ shared_state.market.your_account.shares }}.\n"
195         "Your previous decisions and fills:\n"
196         "{% for h in shared_state.market.your_account.history %}"
197         "Period {{ h.period }}: {{ h.side }} {{ h.decision.quantity }} at limit "
198         "{{ h.limit_cents / 100 }}; admitted {{ h.accepted_quantity }}; signed fill "
199         "{{ h.fill }}; market price {{ h.transaction_price }}; interest "
200         "{{ h.interest }}; dividend per share {{ h.dividend_per_share }}. "
201         f"Your forecasts ({history_labels}): {history_values}. Your rationale: {{{ h.decision.rationale }}}\n\n"

```

```

202     "{% else %}No previous decisions.\n{% endfor %}\n"
203     "PUBLIC HISTORY\n"
204     "{% for row in shared_state.market.tape %}"
205     "Period {{ row.period }}: transaction price {{ row.price }}, volume "
206     "{{ row.volume }}, dividend per share {{ row.dividend }}, total cash "
207     "{{ row.total_cash }}, total shares {{ row.total_shares }}, total interest "
208     "{{ row.total_interest }}.\n"
209     "{% else %}The market has not traded yet. There is no opening reference quote.\n{% endfor %}"
210     "A null or None price means there was no transaction that period.\n\n"
211     "YOUR DECISION\n"
212     + forecast_instruction
213     + "your order using your trading approach. For forecast horizons beyond the "
214     "last trading period, use the terminal redemption amount. Return "
215     f"{{forecast_names}}, side ('buy', 'sell', or 'hold'), price, "
216     "integer quantity, and a brief rationale. All forecasts and prices are in "
217     "dollars. For hold use price=0 and quantity=0. Retain any precommitted "
218     "schedule or exit plan in your rationale.\n"
219 )

```

B.4 Agents, market, and execution plan

```

222 def build_agents(rules=RULES, treatment=TREATMENT):
223     """Assign the declared composition reproducibly; render economic persona facts."""
224     types = list(treatment.composition)
225     random.Random(treatment.assignment_seed).shuffle(types)
226     agents = [
227         Agent(
228             name=f"trader-{seat:02d}",
229             traits={"role": "trader", "seat": seat},
230             instruction=COMMON
231             + PERSONAS[kind].format(cash_return=f"{rules.interest.rate * 100:g}%"),
232         )
233         for seat, kind in enumerate(types)
234     ]
235     agents.append(Agent(name="exchange", traits={"role": "exchange"}))
236     return agents
237
238
239 def build_market(trader_names, rules=RULES, seed=SEED):
240     return call_market(trader_names, rules=rules, seed=seed)
241
242
243 def build_execution_plan(treatment=TREATMENT):
244     """EDSL asks traders; the exchange receives a literal scripted answer."""
245     return (
246         ExecutionPlan()
247         .bind(
248             role("trader"),
249             llm(
250                 model=treatment.model,
251                 service=treatment.service,
252                 parameters={
253                     "temperature": treatment.temperature,
254                     "reasoning_effort": treatment.reasoning_effort,
255                     "max_tokens": treatment.max_tokens,
256                 },
257             ),
258         )
259         .bind(role("exchange"), scripted(answers={"settlement": "clear"}))
260     )

```

B.5 Thirty-round workflow

```

263 def build(*, rules=RULES, treatment=TREATMENT, seed=SEED, created_at=None):
264     """Build a serializable experiment: state, participants, workflow, execution."""
265     if rules.forecast_keys != treatment.decision_schema.forecast_keys:
266         raise ValueError("market forecast_keys must match treatment forecast_horizons")

```

```

267 agents = build_agents(rules, treatment)
268 machine = build_market(
269     [a.name for a in agents if a.traits["role"] == "trader"], rules, seed
270 )
271 states = SharedStateMap(
272     SharedState(market=machine), state_id="portable-asset-market"
273 )
274 market = states.by("session").market
275 workflow = Workflow(
276     f"Speculative asset market: full {rules.periods}-round session",
277     metadata={
278         "economic_horizon": rules.periods,
279         "prompt_variant": "speculative-v1",
280         "observation_design": f"Full {rules.periods} rounds fixed before execution; no early observation
281         pauses.",
282     },
283 )
284 previous = None
285 for period in range(1, rules.periods + 1):
286     question = treatment.decision_schema.question(
287         question_name="decision",
288         question_text=question_text(period, rules, treatment),
289     )
290     orders = workflow.step(
291         f"orders-{period}",
292         Survey([question]),
293         assigned_to=role("trader"),
294         after=previous,
295         visible_to=role("exchange"),
296         reads=(market.read(),),
297         writes=(
298             market.submit(
299                 trader=current.agent.name, period=period, decision=question.answer
300             ),
301         ),
302         metadata={"period": period, "phase": "sealed orders"},
303     )
304     previous = workflow.step(
305         f"settle-{period}",
306         Survey(
307             [
308                 QuestionFreeText(
309                     question_name="settlement",
310                     question_text="Run deterministic market settlement.",
311                 )
312             ]
313         ),
314         assigned_to=role("exchange"),
315         after=orders,
316         writes=(market.settle(period=period),),
317         metadata={"period": period},
318     )
319 return WorkflowExperiment(
320     workflow.compile(),
321     [states],
322     agents,
323     build_execution_plan(treatment),
324     metadata={
325         "seed": seed,
326         "prompt_variant": "speculative-v1",
327         "economic_horizon": rules.periods,
328         "observation_horizon": rules.periods,
329         "fresh_model_calls": True,
330         "created_at": created_at or datetime.now(timezone.utc).isoformat(),
331     },
332 )

```

B.6 Command-line entry point

```

334 def main():
335     parser = argparse.ArgumentParser(description=__doc__)
336     parser.add_argument("--output", type=Path, required=True)
337     parser.add_argument(
338         "--responses", type=Path, help="Saved answers; prevents model calls"
339     )
340     parser.add_argument("--rules", type=Path, help="CallMarketRules JSON")
341     parser.add_argument("--treatment", type=Path, help="TraderTreatment JSON")
342     parser.add_argument("--seed", type=int, default=SEED)
343     parser.add_argument("--resume", action="store_true")
344     parser.add_argument("--prepare-only", action="store_true")
345     args = parser.parse_args()
346     rules = (
347         CallMarketRules.from_dict(json.loads(args.rules.read_text()))
348         if args.rules
349         else RULES
350     )
351     treatment = (
352         TraderTreatment.from_dict(json.loads(args.treatment.read_text()))
353         if args.treatment
354         else TREATMENT
355     )
356     specification = args.output / "specification.json"
357     if (args.resume or specification.exists()) and (
358         args.rules or args.treatment or args.seed != SEED
359     ):
360         raise ValueError(
361             "cannot override a saved specification; use a fresh output directory"
362         )
363     if args.prepare_only:
364         if specification.exists():
365             raise FileExistsError(specification)
366         experiment = build(rules=rules, treatment=treatment, seed=args.seed)
367         args.output.mkdir(parents=True, exist_ok=True)
368         experiment.save(specification)
369         print(json.dumps({"status": "prepared", "path": str(specification)}))
370         return
371     if args.resume:
372         experiment = WorkflowExperiment.load(args.output / "experiment.json")
373     elif specification.exists():
374         experiment = WorkflowExperiment.load(specification)
375     else:
376         experiment = build(rules=rules, treatment=treatment, seed=args.seed)
377     responses = None
378     if args.responses:
379         responses = json.loads(args.responses.read_text())
380     else:
381         # Hosted execution uses locally configured provider credentials.
382         from dotenv import load_dotenv
383
384         load_dotenv()
385     result = experiment.run(args.output, responses=responses, resume=args.resume)
386     (args.output / "completion.json").write_text(json.dumps(result, indent=2) + "\n")
387     from edsl.sharedstate import SQLiteStateBackend
388
389     state = (
390         SQLiteStateBackend(experiment.states[0], args.output / "state-0.sqlite")
391         .snapshot("session")
392         .state["market"]
393     )
394     (args.output / "market.json").write_text(json.dumps(state, indent=2) + "\n")
395     periods = experiment.metadata["economic_horizon"]
396     traders = sum(a.traits["role"] == "trader" for a in experiment.agents)
397     assert (
398         result["status"] == "completed"
399         and result["completed_items"] == (traders + 1) * periods
400     )
401     assert state["finished"] and len(state["tape"]) == periods
402     assert len(state["order_log"]) == traders * periods
403     assert all(a["shares"] == 0 for a in state["accounts"].values())
404     print(json.dumps({"status": "ok", "data": result, "warnings": []}))
405
406

```

```

407 if __name__ == "__main__":
408     main()

```

C Validated economic rules

This is a new file; all its lines are highlighted. Configuration objects reject unknown fields and invalid nested values when constructed.

Source: code/market_rules.py.

C.1 Base model, alternatives, and income objects

```

1  """Validated authoring objects for the version-1 call market.
2
3  These objects contain data only. The wire format remains the original Machine
4  constants; runtime implementation identifiers are not experimental treatments.
5  """
6
7  from enum import Enum
8  import keyword
9  import math
10 from typing import Annotated, Literal, Union
11
12 from pydantic import (
13     BaseModel,
14     ConfigDict,
15     Field,
16     StrictFloat,
17     StrictInt,
18     StrictStr,
19     field_validator,
20     model_validator,
21 )
22
23 NonnegativeInt = Annotated[StrictInt, Field(ge=0)]
24 PositiveInt = Annotated[StrictInt, Field(gt=0)]
25 NonnegativeNumber = Annotated[StrictFloat, Field(ge=0, allow_inf_nan=False)]
26 Name = Annotated[StrictStr, Field(min_length=1)]
27
28
29 class MarketModel(BaseModel):
30     model_config = ConfigDict(
31         extra="forbid", frozen=True, validate_default=True, allow_inf_nan=False
32     )
33
34     def to_dict(self):
35         return self.model_dump(mode="json")
36
37     @classmethod
38     def from_dict(cls, data):
39         return cls.model_validate(data)
40
41     def with_changes(self, **changes):
42         """Unlike model_copy(update=...), revalidate every proposed change."""
43         return type(self).model_validate(**self.model_dump(), **changes)
44
45
46 class PricingRule(str, Enum):
47     MARGINAL_MIDPOINT = "marginal_midpoint"
48     MARGINAL_BID = "marginal_bid"
49     MARGINAL_ASK = "marginal_ask"
50
51
52 class DividendSampling(str, Enum):
53     CHOICE = "python_random_choice_v1"
54     WEIGHTED = "python_random_choices_v1"
55

```

```

56
57 class Endowment(MarketModel):
58     cash_cents: NonnegativeInt = 10_000
59     shares: NonnegativeInt = 4
60
61
62 class Redemption(MarketModel):
63     value_cents: NonnegativeInt = 1_400
64
65
66 class DiscreteDistribution(MarketModel):
67     values_cents: tuple[NonnegativeInt, ...] = (40, 100)
68     probabilities: tuple[NonnegativeNumber, ...] = (0.5, 0.5)
69
70     @field_validator("values_cents")
71     @classmethod
72     def nonempty(cls, values):
73         if not values:
74             raise ValueError("must contain at least one dividend")
75         return values
76
77     @field_validator("probabilities")
78     @classmethod
79     def probability_vector(cls, probabilities, info):
80         values = info.data.get("values_cents")
81         if values is not None and len(values) != len(probabilities):
82             raise ValueError(
83                 f"expected {len(values)} probabilities matching values_cents; received {len(probabilities)}"
84             )
85         if not math.isclose(sum(probabilities), 1):
86             raise ValueError("dividend probabilities must sum to one")
87         return probabilities
88
89     @property
90     def expected_cents(self):
91         return sum(v * p for v, p in zip(self.values_cents, self.probabilities))
92
93
94 class CashInterest(MarketModel):
95     kind: Literal["cash_interest"] = "cash_interest"
96     rate: NonnegativeNumber = 0.05
97
98
99 class ShareDividend(MarketModel):
100     kind: Literal["share_dividend"] = "share_dividend"
101     distribution: DiscreteDistribution = Field(default_factory=DiscreteDistribution)
102     # Retain the exact random algorithm for archived v1 replays. None chooses
103     # the appropriate implementation once, when authoring the specification.
104     sampling: DividendSampling | None = None
105
106     @model_validator(mode="after")
107     def sampling_matches_distribution(self):
108         p = self.distribution.probabilities
109         equal = all(math.isclose(x, 1 / len(p)) for x in p)
110         if self.sampling == DividendSampling.CHOICE and not equal:
111             raise ValueError("choice sampling requires equal probabilities")
112         if self.sampling is None:
113             object.__setattr__(
114                 self,
115                 "sampling",
116                 DividendSampling.CHOICE if equal else DividendSampling.WEIGHTED,
117             )
118         return self
119
120     def to_wire(self):
121         return {
122             "kind": self.kind,
123             **self.distribution.to_dict(),
124             "sampling": self.sampling.value,
125         }
126
127
128 IncomeRule = Annotated[Union[CashInterest, ShareDividend], Field(discriminator="kind")]

```

C.2 Market rules and cross-field validation

```

131 class CallMarketRules(MarketModel):
132     periods: PositiveInt = Field(
133         30, description="Number of trading and income periods."
134     )
135     endowment: Endowment = Field(default_factory=Endowment)
136     initial_reference_price: NonnegativeNumber | None = 14.0
137     redemption: Redemption = Field(default_factory=Redemption)
138     pricing: PricingRule = PricingRule.MARGINAL_MIDPOINT
139     maximum_buy_quantity: PositiveInt | None = None
140     income: tuple[IncomeRule, ...] = Field(
141         default_factory=lambda: (CashInterest(), ShareDividend())
142     )
143     forecast_keys: tuple[Name, ...] = (
144         "forecast_0",
145         "forecast_2",
146         "forecast_5",
147         "forecast_10",
148     )
149
150     @field_validator("income")
151     @classmethod
152     def one_of_each(cls, income):
153         if len(income) != 2 or {type(r) for r in income} != {
154             CashInterest,
155             ShareDividend,
156         }:
157             raise ValueError("income must order one interest and one dividend payment")
158         return income
159
160     @field_validator("forecast_keys")
161     @classmethod
162     def distinct_forecasts(cls, keys):
163         if any(
164             not k.isidentifier()
165             or k.startswith("_")
166             or keyword.iskeyword(k)
167             or hasattr(MarketModel, k)
168             for k in keys
169         ):
170             raise ValueError(
171                 "forecast_keys must be public identifiers that do not shadow model attributes"
172             )
173         if len(keys) != len(set(keys)) or set(keys) & {
174             "side",
175             "price",
176             "quantity",
177             "rationale",
178         }:
179             raise ValueError(
180                 "forecast_keys must be distinct and cannot overlap order fields"
181             )
182         return keys
183
184     @property
185     def interest(self):
186         return next(r for r in self.income if isinstance(r, CashInterest))
187
188     @property
189     def dividend(self):
190         return next(r for r in self.income if isinstance(r, ShareDividend))

```

C.3 Valuation and economic instructions

```

192 def fundamental_value(self, period=1):
193     """Pre-trade risk-neutral value in dollars, ignoring cent rounding.
194
195     Backward induction uses the actual payment order and terminal value;
196     it also works at zero interest and when value changes over time.
197     """

```

```

198     if (
199         isinstance(period, bool)
200         or not isinstance(period, int)
201         or not 1 <= period <= self.periods + 1
202     ):
203         raise ValueError("period must be between 1 and periods + 1")
204     rate = self.interest.rate
205     dividend = self.dividend.distribution.expected_cents / 100
206     if isinstance(self.income[0], ShareDividend):
207         dividend *= 1 + rate
208     value = self.redemption.value_cents / 100
209     for _ in range(self.periods - period + 1):
210         value = (value + dividend) / (1 + rate)
211     return value
212
213 def instructions(self, trader_count):
214     """Economic rules for trader prompts, generated from executed settings."""
215     if (
216         isinstance(trader_count, bool)
217         or not isinstance(trader_count, int)
218         or trader_count < 1
219     ):
220         raise ValueError("trader_count must be a positive integer")
221     d = self.dividend.distribution
222     if len(d.values_cents) == 2 and d.proBABILITIES == (0.5, 0.5):
223         distribution = (
224             " or ".join(f"${v / 100:.2f}" for v in d.values_cents)
225             + ", equally likely"
226         )
227     else:
228         distribution = ", ".join(
229             f"${v / 100:.2f} (probability {p:g})"
230             for v, p in zip(d.values_cents, d.proBABILITIES)
231         )
232     income = [
233         f"cash earns {r.rate * 100:g}% interest"
234         if isinstance(r, CashInterest)
235         else f"each held share pays the same dividend: {distribution}"
236         for r in self.income
237     ]
238     extra = ""
239     if self.pricing != PricingRule.MARGINAL_MIDPOINT:
240         boundary = "bid" if self.pricing == PricingRule.MARGINAL_BID else "ask"
241         extra += f" The clearing price is the last matched {boundary}."
242     if (
243         self.maximum_buy_quantity is not None
244         and self.maximum_buy_quantity < trader_count * self.endowment.shares
245     ):
246         extra += (
247             f" Each buy order is also capped at {self.maximum_buy_quantity} shares."
248         )
249     return (
250         f"There are {trader_count} traders. Each began with ${self.endowment.cash_cents / 100:g} cash and {
251             self.endowment.shares} shares. Submit one "
252         "sealed limit order: buy, sell, or hold. A buy limit is the most you will "
253         "pay per share; a sell limit is the least you will accept. The exchange "
254         "matches crossing orders and all filled orders receive one clearing price. "
255         "Unmatched orders expire. You cannot borrow or short shares. Buy quantities "
256         "are capped by cash divided by your limit; sells by your inventory. Quotes "
257         f"round to cents.{extra} After trading, " + ", then ".join(income) + ". "
258         f"After period {self.periods}'s income, each remaining share is redeemed for ${self.redemption.
259         value_cents / 100:g}; you keep all cash.\n\n"
260     )

```

C.4 Serialization and legacy authoring compatibility

```

260 def to_constants(self, trader_count, seed):
261     if (
262         isinstance(trader_count, bool)
263         or not isinstance(trader_count, int)
264         or trader_count < 1
265     ):

```

```

266         raise ValueError("trader_count must be a positive integer")
267     total = self.endowment.shares * trader_count
268     data = dict(
269         periods=self.periods,
270         seed=seed,
271         total_shares=total,
272         maximum_buy_quantity=self.maximum_buy_quantity or max(1, total),
273         redemption_cents=self.redemption.value_cents,
274         pricing_rule=self.pricing.value,
275         rounding="half_up",
276         money_decimals=2,
277         borrowing=False,
278         short_selling=False,
279         tie_breaking="seeded_price_priority_v1",
280         unfilled_orders="expire",
281         redemption_timing="after_final_income",
282         forecast_keys=list(self.forecast_keys),
283         income_schedule=[
284             r.to_wire() if isinstance(r, ShareDividend) else r.to_dict()
285             for r in self.income
286         ],
287     )
288     MarketConstants.model_validate(data)
289     return data
290
291 @classmethod
292 def from_legacy(cls, **options):
293     """Validate the original keyword API through the same typed rules."""
294     aliases = {
295         "initial_cash_cents",
296         "initial_shares",
297         "redemption_cents",
298         "pricing_rule",
299         "income_schedule",
300     }
301     allowed = {
302         "periods",
303         "initial_reference_price",
304         "maximum_buy_quantity",
305         "forecast_keys",
306     } | aliases
307     unknown = set(options) - allowed
308     if unknown:
309         raise ValueError(f"unknown call_market options: {sorted(unknown)}")
310     data = {k: v for k, v in options.items() if k not in aliases}
311     data["endowment"] = {
312         "cash_cents": options.get("initial_cash_cents", 10000),
313         "shares": options.get("initial_shares", 4),
314     }
315     data["redemption"] = {"value_cents": options.get("redemption_cents", 1400)}
316     data["pricing"] = options.get("pricing_rule", PricingRule.MARGINAL_MIDPOINT)
317     if options.get("income_schedule") is not None:
318         # Validate wire records before reading keys, preserving useful paths.
319         schedule = IncomeSchedule(schedule=options["income_schedule"]).schedule
320         data["income"] = tuple(
321             r.to_authoring()
322             if isinstance(r, WireDividend)
323             else CashInterest(rate=r.rate)
324             for r in schedule
325         )
326     return cls.model_validate(data)

```

C.5 Validation of the existing wire format

```

329 class WireInterest(CashInterest):
330     rate: NonnegativeNumber
331
332
333 class WireDividend(MarketModel):
334     kind: Literal["share_dividend"]
335     values_cents: tuple[NonnegativeInt, ...]
336     probabilities: tuple[NonnegativeNumber, ...]

```

```

337     sampling: DividendSampling
338
339     @model_validator(mode="after")
340     def validate_distribution(self):
341         self.to_authoring()
342         return self
343
344     def to_authoring(self):
345         return ShareDividend(
346             distribution=DiscreteDistribution(
347                 values_cents=self.values_cents, probabilities=self.probabilities
348             ),
349             sampling=self.sampling,
350         )
351
352
353 WireIncome = Annotated[Union[WireInterest, WireDividend], Field(discriminator="kind")]
354
355
356 class IncomeSchedule(MarketModel):
357     schedule: tuple[WireIncome, ...]
358
359     @field_validator("schedule")
360     @classmethod
361     def one_of_each(cls, schedule):
362         if len(schedule) != 2 or {type(r) for r in schedule} != {
363             WireInterest,
364             WireDividend,
365         }:
366             raise ValueError(
367                 "income_schedule must order one interest and one dividend payment"
368             )
369         return schedule
370
371
372 class MarketConstants(MarketModel):
373     """Strict schema for the existing v1 wire format, including fixed policies."""
374
375     periods: PositiveInt
376     seed: StrictStr | StrictInt
377     total_shares: NonnegativeInt
378     maximum_buy_quantity: PositiveInt
379     redemption_cents: NonnegativeInt
380     pricing_rule: PricingRule
381     rounding: Literal["half_up"]
382     money_decimals: Literal[2]
383     borrowing: Literal[False]
384     short_selling: Literal[False]
385     tie_breaking: Literal["seeded_price_priority_v1"]
386     unfilled_orders: Literal["expire"]
387     redemption_timing: Literal["after_final_income"]
388     forecast_keys: tuple[Name, ...]
389     income_schedule: tuple[WireIncome, ...]
390
391     @field_validator("money_decimals", "borrowing", "short_selling", mode="before")
392     @classmethod
393     def exact_literal_type(cls, value, info):
394         expected = int if info.field_name == "money_decimals" else bool
395         if type(value) is not expected:
396             raise ValueError(f"expected {expected.__name__}")
397         return value
398
399     @field_validator("forecast_keys")
400     @classmethod
401     def forecasts(cls, keys):
402         return CallMarketRules.distinct_forecasts(keys)
403
404     @field_validator("income_schedule")
405     @classmethod
406     def income_order(cls, schedule):
407         return IncomeSchedule.one_of_each(schedule)

```

D Decision and market-record contracts

This is a new file. The decision schema generates QuestionDict fields, while runtime validation applies stronger economic and structural constraints.

Source: code/market_records.py.

D.1 Order intent and rejection semantics

```

1  """Call-market decision contracts and validated wire records.
2
3  Rejected orders retain their original, possibly invalid payload. They are audit
4  records, not valid TraderDecisions. No validator silently repairs a decision.
5  """
6
7  from decimal import Decimal, ROUND_HALF_UP
8  from enum import Enum
9  from functools import lru_cache
10 from typing import Any
11
12 from pydantic import (
13     StrictInt,
14     StrictStr,
15     ValidationError,
16     create_model,
17     model_validator,
18 )
19
20 from .market_rules import (
21     CallMarketRules,
22     MarketModel,
23     Name,
24     NonnegativeInt,
25     NonnegativeNumber,
26     PositiveInt,
27 )
28
29
30 def cents(value):
31     return int(
32         (Decimal(str(value)) * 100).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
33     )
34
35
36 class OrderSide(str, Enum):
37     BUY = "buy"
38     SELL = "sell"
39     HOLD = "hold"
40
41
42 class OrderIntent(MarketModel):
43     side: OrderSide
44     price: NonnegativeNumber
45     quantity: NonnegativeInt
46
47     @model_validator(mode="after")
48     def executable_order(self):
49         if self.side != OrderSide.HOLD and (
50             cents(self.price) <= 0 or self.quantity == 0
51         ):
52             raise ValueError("active orders require positive price and quantity")
53         return self
54
55     @classmethod
56     def rejection(cls, answer):
57         """Preserve v1 rejection messages and order-admission semantics."""
58         try:
59             cls.model_validate({key: answer.get(key) for key in cls.model_fields})
60         except ValidationError as exc:
61             fields = {e["loc"][0] for e in exc.errors() if e["loc"]}
62             for key, message in (

```

```

63         ("side", "invalid side"),
64         ("quantity", "quantity must be a nonnegative integer"),
65         ("price", "price must be finite and nonnegative"),
66     ):
67         if key in fields:
68             return message
69         return "active orders require positive price and quantity"
70     return None
71
72
73 class TraderDecision(OrderIntent):
74     rationale: StrictStr

```

D.2 Shared decision schema

```

77 class DecisionSchema(MarketModel):
78     """Shared field definitions for QuestionDict and runtime validation.
79
80     QuestionDict supports primitive value types. Enum membership, finiteness,
81     positivity and cross-field constraints are enforced here on submission.
82     """
83
84     forecast_keys: tuple[Name, ...] = (
85         "forecast_0",
86         "forecast_2",
87         "forecast_5",
88         "forecast_10",
89     )
90
91     @model_validator(mode="after")
92     def distinct_keys(self):
93         CallMarketRules.distinct_forecasts(self.forecast_keys)
94         return self
95
96     def response_model(self):
97         return _response_model(self.forecast_keys, TraderDecision)
98
99     def validate_decision(self, answer):
100         return self.response_model().model_validate(answer)
101
102     def validate_forecasts(self, answer):
103         model = _response_model(self.forecast_keys, MarketModel)
104         return model.model_validate(
105             {key: answer.get(key) for key in self.forecast_keys}
106         )
107
108     def question(self, *, question_name, question_text):
109         from edsl import QuestionDict
110
111         order_types = {
112             "side": "str",
113             "price": "float",
114             "quantity": "int",
115             "rationale": "str",
116         }
117         return QuestionDict(
118             question_name=question_name,
119             question_text=question_text,
120             answer_keys=[*self.forecast_keys, *TraderDecision.model_fields],
121             value_types=["float"] * len(self.forecast_keys)
122             + [order_types[key] for key in TraderDecision.model_fields],
123             include_comment=False,
124         )
125
126
127 @lru_cache(maxsize=128)
128 def _response_model(keys, base):
129     return create_model(
130         "MarketResponse",
131         __base__=base,
132         **{key: (NonnegativeNumber, ...) for key in keys},
133     )

```

D.3 Admission, fills, accounts, and round records

```

136 class AcceptedOrder(MarketModel):
137     """Admission outcome, including rejected submissions for the audit trail."""
138
139     trader: Name
140     period: PositiveInt
141     decision: dict[str, Any]
142     side: Any # A rejected order must retain the invalid submitted side.
143     limit_cents: NonnegativeInt
144     accepted_quantity: NonnegativeInt
145     rejection: StrictStr | None
146
147     @model_validator(mode="after")
148     def admitted_or_rejected(self):
149         if self.rejection is None:
150             OrderIntent.model_validate(
151                 {k: self.decision.get(k) for k in OrderIntent.model_fields}
152             )
153             if self.side != self.decision["side"] or self.limit_cents != cents(
154                 self.decision["price"]
155             ):
156                 raise ValueError(
157                     "admitted order must match its submitted side and limit"
158                 )
159             if self.accepted_quantity > self.decision["quantity"]:
160                 raise ValueError("admitted quantity exceeds submitted quantity")
161             if self.side == OrderSide.HOLD and self.accepted_quantity:
162                 raise ValueError("hold orders cannot be admitted for trading")
163             elif self.accepted_quantity or self.limit_cents:
164                 raise ValueError(
165                     "rejected orders must have zero admitted quantity and limit"
166                 )
167         return self
168
169
170 class TradeFill(AcceptedOrder):
171     fill: StrictInt
172     transaction_price: NonnegativeNumber | None
173     interest: NonnegativeNumber
174     dividend_per_share: NonnegativeNumber
175
176     @model_validator(mode="after")
177     def fill_matches_order(self):
178         if abs(self.fill) > self.accepted_quantity:
179             raise ValueError("fill exceeds admitted quantity")
180         if self.fill and (
181             self.transaction_price is None
182             or (self.fill > 0 and self.side != OrderSide.BUY)
183             or (self.fill < 0 and self.side != OrderSide.SELL)
184         ):
185             raise ValueError("fill must match order side and have a transaction price")
186         return self
187
188
189 class Account(MarketModel):
190     cash_cents: NonnegativeInt
191     shares: NonnegativeInt
192     history: tuple[TradeFill, ...] = ()
193     redeemed_shares: NonnegativeInt | None = None
194
195     def to_dict(self):
196         exclude = {"redeemed_shares"} if self.redeemed_shares is None else set()
197         return self.model_dump(mode="json", exclude=exclude)
198
199
200 class RoundOutcome(MarketModel):
201     period: PositiveInt
202     price: NonnegativeNumber | None
203     reference_price: NonnegativeNumber | None

```

```

204     volume: NonnegativeInt
205     dividend: NonnegativeNumber
206     total_interest: NonnegativeNumber
207     total_cash: NonnegativeNumber
208     total_shares: NonnegativeInt
209
210     @model_validator(mode="after")
211     def price_requires_trade(self):
212         if (self.volume == 0) != (self.price is None):
213             raise ValueError("price is present exactly when volume is positive")
214         return self

```

E Native market integration

Green lines show changes from the earlier dictionary-based implementation. Clearing economics remain version 1; records and rules now pass through typed validation.

Source: code/call_market.py.

E.1 Factory and serialized machine

```

1  """Version-1 sealed call market: explicit monetary and clearing rules.
2
3  The serialized Machine carries all economic parameters. Implementations are
4  standard runtime capabilities; loading never imports experiment source code.
5  Amounts in state and rule parameters are integer cents, except quoted prices
6  and elicited forecasts, which are in dollars. Random streams are specified by
7  seed and purpose; Python's versioned choice/shuffle behavior is part of v1.
8  """
9
10 from decimal import Decimal, ROUND_HALF_UP
11 import random
12
13 from .dsl import Command, Machine, T, algorithm, current, field, input_, state_field
14 from .market_rules import (
15     CallMarketRules,
16     MarketConstants,
17     WireInterest,
18     WireDividend,
19     PricingRule,
20     DividendSampling,
21 )
22 from .market_records import (
23     Account,
24     AcceptedOrder,
25     DecisionSchema,
26     OrderIntent,
27     TradeFill,
28     RoundOutcome,
29     cents,
30 )
31
32
33 def validate_rules(constants):
34     """Validate original wire data using the same constraints as authoring."""
35     MarketConstants.from_dict(constants)
36
37
38 def call_market(
39     traders,
40     *,
41     rules: CallMarketRules | None = None,
42     seed: str | int = 1,
43     **legacy_options,
44 ) -> Machine:
45     """Build a portable v1 market from validated :class:'CallMarketRules'."""
46

```

```

47 The original keyword arguments remain supported. They cannot be mixed with
48 'rules': an ambiguous override is an authoring error. Income is ordered;
49 terminal redemption follows the final income payment.
50 """
51 traders = list(traders)
52 if (
53     not traders
54     or any(not isinstance(t, str) or not t for t in traders)
55     or len(traders) != len(set(traders))
56 ):
57     raise ValueError("traders must be distinct nonempty names")
58 if rules is not None and legacy_options:
59     raise ValueError("rules cannot be combined with legacy rule keywords")
60 if rules is None:
61     rules = CallMarketRules.from_legacy(**legacy_options)
62 if not isinstance(rules, CallMarketRules):
63     raise TypeError(
64         "rules must be CallMarketRules; use CallMarketRules.from_dict for JSON"
65     )
66 constants = rules.to_constants(len(traders), seed)
67 machine = Machine(
68     name="CallMarket",
69     constants=constants,
70     fields={
71         "accounts": state_field(
72             T.map(),
73             {
74                 t: Account(
75                     cash_cents=rules.endowment.cash_cents,
76                     shares=rules.endowment.shares,
77                 ).to_dict()
78                 for t in traders
79             },
80         ),
81         "orders": state_field(T.map(), {}),
82         "order_log": state_field(T.sequence(), []),
83         "tape": state_field(T.sequence(), []),
84         "period": state_field(T.integer(), 1),
85         "last_price": state_field(
86             T.optional(T.number()), rules.initial_reference_price
87         ),
88         "finished": state_field(T.boolean(), False),
89     },
90     commands={
91         "submit": Command(
92             inputs={"trader": T.text(), "period": T.integer(), "decision": T.map()},
93             effects=(
94                 algorithm(
95                     "call_market_submit",
96                     trader=input_("trader"),
97                     period=input_("period"),
98                     decision=input_("decision"),
99                 ),
100             ),
101         ),
102         "settle": Command(
103             inputs={"period": T.integer()},
104             effects=(algorithm("call_market_settle", period=input_("period")),),
105         ),
106     },
107     view={
108         "period": field("period"),
109         "tape": field("tape"),
110         "last_price": field("last_price"),
111         "your_account": field("accounts").get(current("name")),
112         "finished": field("finished"),
113     },
114     algorithms=("call_market_submit@1", "call_market_settle@1"),
115 )
116 machine.validate()
117 return machine

```

E.2 Order admission

```

120 def submit_order(state, inputs, constants):
121     """Commit a sealed order without changing balances or the public tape."""
122     trader, period, answer = inputs["trader"], inputs["period"], inputs["decision"]
123     if (
124         period != state["period"]
125         or state["finished"]
126         or trader not in state["accounts"]
127     ):
128         raise ValueError("Invalid trader, stale period, or finished market")
129     if trader in state["orders"]:
130         raise ValueError("Trader already submitted this period")
131     account = state["accounts"][trader]
132     rules = MarketConstants.from_dict(constants)
133     DecisionSchema(forecast_keys=rules.forecast_keys).validate_forecasts(answer)
134     side, qty, price = answer.get("side"), answer.get("quantity"), answer.get("price")
135     error = OrderIntent.rejection(answer)
136     limit = cents(price) if error is None else 0
137     accepted_qty = 0
138     if error is None and side == "buy":
139         accepted_qty = min(
140             qty, account["cash_cents"] // limit, rules.maximum_buy_quantity
141         )
142     elif error is None and side == "sell":
143         accepted_qty = min(qty, account["shares"])
144     state["orders"][trader] = AcceptedOrder.from_dict(
145         {
146             "trader": trader,
147             "period": period,
148             "decision": dict(answer),
149             "side": side,
150             "limit_cents": limit,
151             "accepted_quantity": accepted_qty,
152             "rejection": error,
153         }
154     ).to_dict()

```

E.3 Settlement

```

157 def settle_market(state, inputs, constants):
158     """Maximal crossing volume, configured uniform price, then scheduled income."""
159     rules = MarketConstants.from_dict(constants)
160     period = inputs["period"]
161     if period != state["period"] or state["finished"]:
162         raise ValueError("Settlement must occur exactly once per current period")
163     if set(state["orders"]) != set(state["accounts"]):
164         raise ValueError("Cannot clear until every trader has submitted")
165     # A reproducible per-period lottery breaks equal-price ties independently of
166     # completion order. Admission bounds the quantities expanded into unit orders.
167     names = sorted(state["accounts"])
168     random.Random(f"{rules.seed}:priority:{period}").shuffle(names)
169     priority = {name: i for i, name in enumerate(names)}
170     bids, asks = [], []
171     for name, order in state["orders"].items():
172         target = bids if order["side"] == "buy" else asks
173         target.extend(
174             (order["limit_cents"], priority[name], name)
175             for _ in range(order["accepted_quantity"])
176         )
177     bids.sort(key=lambda row: (-row[0], row[1]))
178     asks.sort(key=lambda row: (row[0], row[1]))
179     volume = 0
180     for bid, ask in zip(bids, asks):
181         if bid[0] < ask[0]:
182             break
183         volume += 1
184     price = None
185     if volume:
186         bid, ask = bids[volume - 1][0], asks[volume - 1][0]
187         price = {

```

```

188     PricingRule.MARGINAL_MIDPOINT: (bid + ask + 1) // 2,
189     PricingRule.MARGINAL_BID: bid,
190     PricingRule.MARGINAL_ASK: ask,
191 }[rules.pricing_rule]
192 cash_before = sum(a["cash_cents"] for a in state["accounts"].values())
193 shares_before = sum(a["shares"] for a in state["accounts"].values())
194 fills = {name: 0 for name in names}
195 for bid, ask in zip(bids[:volume], asks[:volume]):
196     buyer, seller = state["accounts"][bid[2]], state["accounts"][ask[2]]
197     buyer["cash_cents"] -= price
198     seller["cash_cents"] += price
199     buyer["shares"] += 1
200     seller["shares"] -= 1
201     fills[bid[2]] += 1
202     fills[ask[2]] -= 1
203 assert sum(a["cash_cents"] for a in state["accounts"].values()) == cash_before
204 assert (
205     sum(a["shares"] for a in state["accounts"].values())
206     == shares_before
207     == rules.total_shares
208 )
209 dividend_rule = next(
210     r for r in rules.income_schedule if isinstance(r, WireDividend)
211 )
212 rng = random.Random(f"{rules.seed}:dividend:{period}")
213 values = dividend_rule.values_cents
214 dividend = (
215     rng.choice(values)
216     if dividend_rule.sampling == DividendSampling.CHOICE
217     else rng.choices(values, weights=dividend_rule.probabilities, k=1)[0]
218 )
219 total_interest = 0
220 for name, account in state["accounts"].items():
221     interest = 0
222     for rule in rules.income_schedule:
223         if isinstance(rule, WireInterest):
224             interest = int(
225                 (Decimal(account["cash_cents"]) * Decimal(str(rule.rate))).quantize(
226                     Decimal("1"), rounding=ROUND_HALF_UP
227                 )
228             )
229             account["cash_cents"] += interest
230             total_interest += interest
231         else:
232             account["cash_cents"] += dividend * account["shares"]
233     assert account["cash_cents"] >= 0 and account["shares"] >= 0
234     account["history"].append(
235         TradeFill.from_dict(
236             {
237                 **state["orders"][name],
238                 "fill": fills[name],
239                 "transaction_price": None if price is None else price / 100,
240                 "interest": interest / 100,
241                 "dividend_per_share": dividend / 100,
242             }
243         ).to_dict()
244     )
245 if price is not None:
246     state["last_price"] = price / 100
247 row = RoundOutcome.from_dict(
248     {
249         "period": period,
250         "price": None if price is None else price / 100,
251         "reference_price": state["last_price"],
252         "volume": volume,
253         "dividend": dividend / 100,
254         "total_interest": total_interest / 100,
255         "total_cash": sum(a["cash_cents"] for a in state["accounts"].values())
256         / 100,
257         "total_shares": shares_before,
258     }
259 ).to_dict()
260 assert (
261     cents(row["total_cash"])
262     == cash_before + total_interest + dividend * shares_before
263 )

```

```

264     state["tape"].append(row)
265     state["order_log"].extend(state["orders"][name] for name in sorted(names))
266     state["orders"] = {}
267     if period == rules.periods:
268         for account in state["accounts"].values():
269             account["redeemed_shares"] = account["shares"]
270             account["cash_cents"] += rules.redemption_cents * account["shares"]
271             account["shares"] = 0
272             state["finished"] = True
273     state["period"] += 1

```

F Complete workflow executor after the retry fix

The baseline is the executor printed in the previous report. The highlighted null-response guard was added after the failed final-round batch; both runtime versions are in the bundle.

Source: code/workflow_experiment.py.

F.1 Bundle and serialization

```

1  """Portable workflow, state, participant, and executor specifications.
2
3  Only data is loaded. Algorithm implementations must already be available in
4  the selected EDSL runtime; no source text, imports, or pickle is executed.
5  """
6
7  from copy import deepcopy
8  import json
9  from pathlib import Path
10
11  from edsl import Agent, Model, Scenario, Survey
12  from edsl._data_contracts import definition_fingerprint, validate_data
13  from edsl.sharedstate import SharedStateMap, SQLiteStateBackend
14  from edsl.sharedstate.dsl_runtime import default_runtime
15
16  from .coordinator import WorkflowCoordinator
17  from .definition import HumanWorkflow
18  from .execution import ExecutionPlan
19  from .store import SQLiteWorkflowStore
20
21
22  class WorkflowExperiment:
23      """A JSON-portable experiment with explicit runtime capability dependencies."""
24
25      def __init__(
26          self, workflow, states, agents, execution_plan, *, metadata=None, runtime=None
27      ):
28          agents = tuple(agents)
29          if any(
30              a.has_dynamic_traits_function or hasattr(a, "answer_question_directly")
31              for a in agents
32          ):
33              raise ValueError("portable agents cannot contain Python callbacks")
34          self._check_questions(workflow.to_dict())
35          self.workflow = HumanWorkflow.from_dict(workflow.to_dict())
36          self.states = tuple(SharedStateMap.from_dict(s.to_dict()) for s in states)
37          self.agents = tuple(Agent.from_dict(a.to_dict()) for a in agents)
38          self.execution_plan = ExecutionPlan.from_dict(execution_plan.to_dict())
39          self.metadata = deepcopy(metadata or {})
40          self.runtime = runtime or default_runtime()
41          self.validate()
42
43      def validate(self):
44          validate_data(self.to_dict(), path="workflow experiment")
45          maps = {s.state_id: s for s in self.states}
46          if len(maps) != len(self.states):
47              raise ValueError("experiment state IDs must be unique")

```

```

48 names = [a.name for a in self.agents]
49 if any(not n for n in names) or len(names) != len(set(names)):
50     raise ValueError("experiment agents require unique names")
51 for state in self.states:
52     for machine in state.definition.machines.values():
53         self.runtime.validate_capabilities(machine)
54 operations = [
55     op for step in self.workflow.steps for op in (*step.reads, *step.writes)
56 ]
57 operations += [rule.read for rule in self.workflow.pause_rules]
58 for op in operations:
59     if op.state_id not in maps or definition_fingerprint(
60         op.definition.to_dict()
61     ) != definition_fingerprint(maps[op.state_id].definition.to_dict()):
62         raise ValueError(
63             "workflow references an absent or different state definition"
64         )
65 for step in self.workflow.steps:
66     if not any(step.assignee.matches(a.traits) for a in self.agents):
67         raise ValueError(f"step {step.name!r} has no assigned participant")
68 for agent in self.agents:
69     executor = self.execution_plan.resolve(agent.traits)
70     if executor.kind == "llm" and not executor.options.get("model"):
71         raise ValueError("LLM executor must name a model")
72     if executor.kind == "scripted" and not isinstance(
73         executor.options.get("answers"), dict
74     ):
75         raise ValueError("portable scripted executor requires literal answers")
76
77 def to_dict(self):
78     return {
79         "type": "workflow_experiment",
80         "version": 1,
81         "workflow": self.workflow.to_dict(),
82         "states": [s.to_dict() for s in self.states],
83         "agents": [a.to_dict() for a in self.agents],
84         "execution_plan": self.execution_plan.to_dict(),
85         "metadata": deepcopy(self.metadata),
86     }
87
88 @classmethod
89 def from_dict(cls, data, *, runtime=None):
90     if data.get("type") != "workflow_experiment" or data.get("version") != 1:
91         raise ValueError("unsupported workflow experiment serialization")
92     cls._check_questions(data["workflow"])
93     if any("traits" not in agent for agent in data["agents"]):
94         raise ValueError(
95             "portable agents require the data-only traits representation"
96         )
97     return cls(
98         HumanWorkflow.from_dict(data["workflow"]),
99         [SharedStateMap.from_dict(s) for s in data["states"]],
100         [Agent.from_dict(deepcopy(a)) for a in data["agents"]],
101         ExecutionPlan.from_dict(data["execution_plan"]),
102         metadata=data.get("metadata"),
103         runtime=runtime,
104     )
105
106 @staticmethod
107 def _check_questions(workflow):
108     for step in workflow["steps"]:
109         for question in step["survey"]["questions"]:
110             if question.get("question_type") == "functional":
111                 raise ValueError(
112                     "portable experiments do not embed functional Python questions"
113                 )
114
115 def save(self, path):
116     path = Path(path)
117     temporary = path.with_name(path.name + ".tmp")
118     temporary.write_text(
119         json.dumps(self.to_dict(), indent=2, allow_nan=False) + "\n"
120     )
121     temporary.replace(path)
122
123 @classmethod

```

```

124 def load(cls, path, *, runtime=None):
125     return cls.from_dict(json.loads(Path(path).read_text()), runtime=runtime)

```

F.2 Execution and recovery

```

127 def run(self, output, *, responses=None, resume=False, instance_id="experiment"):
128     """Execute until a declared pause or completion.
129
130     'responses' is optional replay data: a sequence of dictionaries with
131     'step', 'participant', and 'answers'. Supplying it forbids model
132     calls, including when an answer is missing. Without it, configured LLM
133     executors call their hosted provider through EDSL. All questions in a
134     ready batch are opened before submitting any of that batch's answers.
135     Live LLM work in each ready batch is executed as one EDSL Jobs object,
136     preserving each interview's own rendered survey, agent, and model.
137
138     """
139     output = Path(output)
140     pinned = output / "experiment.json"
141     if pinned.exists():
142         if not resume:
143             raise FileExistsError("saved experiment exists; use resume=True")
144         if json.loads(pinned.read_text()) != self.to_dict():
145             raise ValueError("saved experiment specification changed")
146     elif resume:
147         raise FileNotFoundError("no experiment to resume")
148     replay = None
149     if responses is not None:
150         replay = {}
151         for row in responses:
152             key = (row["step"], row["participant"])
153             if key in replay:
154                 raise ValueError(f"duplicate replay response: {key}")
155             replay[key] = row["answers"]
156     self.validate()
157     output.mkdir(parents=True, exist_ok=True)
158     if not pinned.exists():
159         self.save(pinned)
160     store = SQLiteWorkflowStore(output / "workflow.sqlite")
161     backends = {
162         s.state_id: SQLiteStateBackend(
163             s, output / f"state-{i}.sqlite", runtime=self.runtime
164         )
165         for i, s in enumerate(self.states)
166     }
167     if resume:
168         coordinator = WorkflowCoordinator.restore(
169             instance_id, store, state_backends=backends
170         )
171         coordinator.recover(instance_id, max_attempts=3)
172         if store.instance_status(instance_id) == "paused":
173             coordinator.resume(instance_id)
174     else:
175         coordinator = WorkflowCoordinator(
176             self.workflow, store, state_backends=backends
177         )
178         coordinator.launch(
179             self.agents,
180             instance_id=instance_id,
181             random_seed=self.metadata.get("seed", instance_id),
182         )
183     agents = {a.name: a for a in self.agents}
184     while pending := store.pending_outbox(instance_id):
185         opened_items = []
186         for row in pending:
187             store.mark_delivered(row["id"])
188             opened_items.append(coordinator.open(row["work_item_id"]))
189         if replay is None:
190             llm_items = [
191                 item
192                 for item in opened_items
193                 if self.execution_plan.resolve(
194                     agents[item.participant_id].traits

```

```

194         ).kind
195         == "llm"
196     ]
197     groups = {}
198     for item in llm_items:
199         options = self.execution_plan.resolve(
200             agents[item.participant_id].traits
201         ).options
202         key = (
203             item.step_name,
204             json.dumps(options, sort_keys=True),
205             json.dumps(self._batch_survey(item).to_dict(), sort_keys=True),
206         )
207         groups.setdefault(key, []).append(item)
208     for group in groups.values():
209         self._run_llm_batch(group, agents, coordinator, store, output)
210     opened_items = [item for item in opened_items if item not in llm_items]
211     for opened in opened_items:
212         # Scripted work and supplied replay answers need no inference job.
213         attempt = store.start_attempt(opened.id, lease_seconds=600)
214         agent = agents[opened.participant_id]
215         executor = self.execution_plan.resolve(agent.traits)
216         store.record_executor(opened.id, executor.kind, executor.options)
217         try:
218             if executor.kind == "scripted":
219                 answers = deepcopy(executor.options["answers"])
220             elif replay is not None:
221                 answers = deepcopy(replay[(opened.step_name, agent.name)])
222             else:
223                 raise ValueError(
224                     "human work needs a human delivery adapter or replay responses"
225                 )
226             coordinator.submit(
227                 opened.id,
228                 answers,
229                 idempotency_key=f"experiment:{opened.id}",
230                 attempt_id=attempt["id"],
231             )
232         except Exception as exc:
233             if store.item(opened.id)["status"] != "committing":
234                 store.finish_attempt(
235                     attempt["id"],
236                     status="failed",
237                     error_kind="exception",
238                     error_message=str(exc)[:1000],
239                 )
240             store.retry_item(
241                 opened.id, reason="resume after executor failure"
242             )
243             raise
244     status = store.instance_status(instance_id)
245     if status not in {"paused", "completed"}:
246         raise RuntimeError("workflow has unfinished work without a declared pause")
247     return {
248         "status": status,
249         "instance_id": instance_id,
250         "completed_items": sum(
251             i["status"] == "completed" for i in store.items(instance_id)
252         ),
253         "pauses": [
254             e for e in store.events(instance_id) if e["kind"] == "workflow.paused"
255         ],
256     }

```

F.3 Paired EDSL jobs and response admission

```

258     @staticmethod
259     def _batch_survey(item):
260         """Parameterize text only; differing response schemas get separate jobs."""
261         data = item.survey.to_dict()
262         for question in data["questions"]:
263             if "question_name" in question:

```

```

264         name = question["question_name"]
265         question["question_text"] = (
266             "{{ workflow_question_texts[" + repr(name) + " ] }}"
267         )
268     data["memory_plan"]["survey_question_texts"] = [
269         q["question_text"] for q in data["questions"] if "question_name" in q
270     ]
271     return Survey.from_dict(data)
272
273 def _run_llm_batch(self, opened_items, agents, coordinator, store, output):
274     """One EDSL job per compatible step; match unordered results by participant."""
275     from datetime import datetime, timezone
276     import time
277
278     attempts, participants, scenarios = {}, [], []
279     by_id = {item.id: item for item in opened_items}
280     started = time.monotonic()
281     try:
282         for item in opened_items:
283             attempts[item.id] = store.start_attempt(item.id, lease_seconds=600)
284             agent = agents[item.participant_id]
285             executor = self.execution_plan.resolve(agent.traits)
286             options = executor.options
287             store.record_executor(item.id, executor.kind, options)
288             model = Model(
289                 options["model"],
290                 service_name=options.get("service"),
291                 **options.get("parameters", {}),
292             )
293             store.record_model(item.id, model.to_dict())
294             participants.append(agent)
295             scenarios.append(
296                 Scenario(
297                     {
298                         "workflow_work_item_id": item.id,
299                         "workflow_question_texts": {
300                             q.question_name: q.question_text
301                             for q in item.survey.questions
302                         },
303                     }
304                 )
305             )
306         if len({a.name for a in participants}) != len(opened_items):
307             raise ValueError("batch requires distinct participants")
308         # Pair agent i only with its own frozen private scenario i.
309         job = (
310             self._batch_survey(opened_items[0])
311             .by(participants)
312             .by(scenarios)
313             .by(model)
314             .zip_assign(over=("agents", "scenarios"))
315         )
316         jobs_folder = output / "jobs"
317         jobs_folder.mkdir(exist_ok=True)
318         job_path = (
319             jobs_folder
320             / f"batch-{opened_items[0].id}-{attempts[opened_items[0].id]['id']}.json"
321         )
322         job_path.write_text(json.dumps(job.to_dict(), indent=2) + "\n")
323         batch = {
324             "started_at": datetime.now(timezone.utc).isoformat(),
325             "work_item_ids": list(by_id),
326             "steps": sorted({item.step_name for item in opened_items}),
327             "interviews": len(opened_items),
328             "job_path": str(job_path.relative_to(output)),
329         }
330         with (output / "inference-batches.jsonl").open("a") as handle:
331             handle.write(json.dumps({**batch, "event": "started"}) + "\n")
332         results = job.run(
333             disable_remote_inference=True,
334             disable_remote_cache=True,
335             cache=False,
336             stop_on_exceptions=False,
337         )
338         seen, failures = set(), []
339         for result in results:

```

```

340     item_id = result.scenario["workflow_work_item_id"]
341     if item_id not in by_id or item_id in seen:
342         raise ValueError("unexpected or duplicate work-item result")
343     seen.add(item_id)
344     item = by_id[item_id]
345     if result.agent.name != item.participant_id:
346         raise ValueError("result participant differs from work item")
347     record = {
348         "work_item_id": item_id,
349         "step": item.step_name,
350         "participant": item.participant_id,
351         "result": result.to_dict(),
352     }
353     with (output / "model-calls.jsonl").open("a") as handle:
354         handle.write(json.dumps(record, allow_nan=False) + "\n")
355     try:
356         required = {q.question_name for q in item.survey.questions}
357         if not required.issubset(result.answer) or any(
358             result.answer[name] is None for name in required
359         ):
360             raise ValueError("incomplete model result")
361         coordinator.submit(
362             item_id,
363             dict(result.answer),
364             idempotency_key=f"experiment:{item_id}",
365             attempt_id=attempts[item_id]["id"],
366         )
367     except Exception as exc:
368         failures.append(exc)
369     if seen != set(by_id):
370         failures.append(ValueError("EDSL job returned an incomplete batch"))
371     if failures:
372         raise failures[0]
373     with (output / "inference-batches.jsonl").open("a") as handle:
374         handle.write(
375             json.dumps(
376                 {
377                     **batch,
378                     "event": "completed",
379                     "results": len(results),
380                     "elapsed_seconds": time.monotonic() - started,
381                 }
382             )
383             + "\n"
384         )
385     except Exception as exc:
386         # Keep successful submissions; only unfinished interviews retry.
387         for item_id, attempt in attempts.items():
388             if store.item(item_id)["status"] not in {"completed", "committing"}:
389                 store.finish_attempt(
390                     attempt["id"],
391                     status="failed",
392                     error_kind="exception",
393                     error_message=str(exc)[:1000],
394                 )
395         store.retry_item(item_id, reason="resume after EDSL batch failure")
396     raise

```

G Terminal accounts and reproducibility

Trader	Type	Final cash (\$)	Difference (\$)	Redeemed
00	MC	268.66	-380.03	17
01	RA	781.99	+133.30	0
02	NT	781.18	+132.49	0
03	NT	783.71	+135.02	0
04	RA	774.27	+125.58	0
05	OC	283.49	-365.20	16
06	PC	741.72	+93.03	0
07	BT	791.25	+142.56	0
08	MC	340.06	-308.63	15
09	PC	736.90	+88.21	0
10	BT	746.21	+97.52	0
11	OC	754.91	+106.22	0

“Difference” is terminal cash minus \$648.69 passive wealth. “Redeemed” is the share count immediately before redemption; every terminal inventory is zero.

Contents of the companion bundle

- **code/**: the complete printed sources, their prior-report baselines and diffs, both executor versions, the guarded recovery script, and reporting helpers. **data/code-changes.json** records highlighted line numbers.
- **data/**: serialized experiment, economic rules and treatment; 360 replay responses; final market state; round, decision and wealth tables; batch log; recovery and verification records; source manifests; prior-session comparison data; all 361 raw result records in a gzip archive.
- **figures/**: three figures in PDF and PNG. **manifest.json** records SHA-256 hashes for code, data, and figures.

To rebuild the PDF from the bundle, run the command below in its root with a TeX installation containing **listings** and **lstlinebgrd**:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

The appendix includes the full study-specific experiment and core market implementation, not a vendored copy of EDSL. Executing or replaying the experiment requires this branch’s compatible EDSL runtime; the serialized specification alone does not install handlers or dependencies. The supplied run artifacts support inspection without making any new model calls. See the bundle README for the local replay command and provenance details.